

List of Dynamic Programming Problems

Reg Dodds
Department of Computer Science
University of the Western Cape
rdodds@uwc.ac.za

©2016 Reg Dodds

August 11, 2016

Abstract

This list of dynamic programming problems has been gleaned from the Internet. I have tried by best to ascribe the original source, where this makes sense. If you know that you invented some particular problem I will be delighted to attribute it to you.

Draft

Chapter 1

Introduction

DYNAMIC PROGRAMMING PROBLEMS often appear to be infeasible until they are identified as falling into this class. I have listed the problems here in no particular order other than putting the lists of authors in one place.

1.1 Dynamic Programming Archives

1. **Largest Sum Contiguous Subarray** Find a contiguous subarray of a one-dimensional list of integers that has the largest sum.
Solution

1.2 Ten Dynamic programming Questions

Here is the list of 10 dynamic programming questions that are commonly asked by interviewers (I made this list while preparing for interviews):

2. **Dice Throw Problem:** Given n dice each with m faces, numbered from 1 to m , find the number of ways to get sum X . X is the summation of values on each face when all the dice are thrown.
3. **Coin Change:** You are given n types of coin denominations of values $v(1) ; v(2) ; \dots ; v(n)$ (all integers). Assume $v(1) = 1$, so you can always make change for any amount of money C . Give an algorithm which makes change for an amount of money C with as few coins as possible.

4. **Counting Boolean Parenthesizations:** You are given a boolean expression consisting of a string of the symbols 'true', 'false', 'and', 'or', and 'xor'. Count the number of ways to parenthesize the expression such that it will evaluate to true. For example, there is only 1 way to parenthesize 'true and false xor true' such that it evaluates to true.
5. **Subset Sum Problem:** Given a set of non-negative integers, and a value sum, determine if there is a subset of the given set with sum equal to given sum.
6. **Minimum Number of Jumps:** Given an array of integers where each element represents the maximum number of steps that can be made forward from that element, find the minimum number of jumps to reach the end of the array (starting from the first element).
7. **Two-Person Traversal of a Sequence of Cities:** You are given an ordered sequence of n cities, and the distances between every pair of cities. You must partition the cities into two subsequences (not necessarily contiguous) such that person A visits all cities in the first subsequence (in order), person B visits all cities in the second subsequence (in order), and such that the sum of the total distances travelled by A and B is minimized. Assume that person A and person B start initially at the first city in their respective subsequences.
8. **Balanced Partition:** You have a set of n integers each in the range $0 \dots K$. Partition these integers into two subsets such that you minimize $|S_1 - S_2|$, where S_1 and S_2 denote the sums of the elements in each of the two subsets.
9. **Optimal Strategy for a Game:** Consider a row of n coins of values $v(1) \dots v(n)$, where n is even. We play a game against an opponent by alternating turns. In each turn, a player selects either the first or last coin from the row, removes it from the row permanently, and receives the value of the coin. Determine the maximum possible amount of money we can definitely win if we move first.
10. **Maximum Value Contiguous Subsequence:** Given a sequence of n real numbers $A(1) \dots A(n)$, determine a contiguous subsequence $A(i) \dots A(j)$ for which the sum of elements in the subsequence is maximized.

11. **Edit Distance:** Given two text strings A of length n and B of length m, you want to transform A into B with a minimum number of operations of the following types: delete a character from A, insert a character into A, or change some character in A into a new character. The minimal number of such operations required to transform A into B is called the edit distance between A and B.

1.3 Harry He

12. **Maximum Sum of All Sub-arrays** A sub-array has one number of some continuous numbers. Given an integer array with positive numbers and negative numbers, get the maximum sum of all sub-arrays. Time complexity should be $O(n)$. For example, in the array 1, -2, 3, 10, -4, 7, 2, -5, its sub-array 3, 10, -4, 7, 2 has the maximum sum 18.
13. **Maximal Length of Incremental Subsequences** Given an unsorted array, find the max length of subsequence in which the numbers are in incremental order.

For example: If the input array is 7, 2, 3, 1, 5, 8, 9, 6, a subsequence with the most numbers in incremental order is 2, 3, 5, 8, 9 and the expected output is 5.
14. **Edit Distance** Implement a function which gets the edit distance of two input strings. There are three types of edit operations: insertion, deletion and substitution. Edit distance is the minimal number of edit operations to modify a string from one to the other.
15. **Minimal Number of Coins for Change** Please implement a function which gets the minimal number of coins, whose value is $v_1, v_2, \dots, v_n$, to make change for an amount of money with value t. Any coin with value v_i may duplicate for any times to make change. For example, the minimal number of coins to make change for 15 out of a set of coins with value 1, 3, 9, 10 is 3. We can choose two coins with value 3 and a coin with value 9. The number of coins for other choices should be greater than 3.
16. **Dynamic Programming on Stolen Values** There are n houses built in a line, each of which contains some value in it. A thief is going to steal the maximal value in these houses, but he cannot steal in two adjacent houses because the owner of a stolen house will tell his two neighbors on the left and right side. What is the maximal stolen value?

17. **Longest Substring without Duplication** Given a string, please get the length of the longest substring which does not have duplicated characters. Supposing all characters in the string are in the range from a to z.
18. **Maximal Product when Cutting Rope** Given a rope with length n , how to cut the rope into m parts with length $n[0], n[1], \dots, n[m-1]$, in order to get the maximal product of $n[0]*n[1]*\dots*n[m-1]$? We have to cut once at least. Additionally, the length of the whole length of the rope, as well as the length of each part, are in integer value.

1.4 Anant Sharma, Spectator

72.9k Views

Following questions are the most popular dynamic programming problems for interviews :

19. **Submatrix of ones only** Given a matrix consisting of 0's and 1's, find the maximum size sub-matrix consisting of only 1's.
20. **Contiguous array with maximum sum** Given an array containing both positive and negative integers, find the contiguous array with the maximum sum.
21. **Longest Increasing Subsequence** Find the length of the longest subsequence of a given sequence such that all the elements are sorted in increasing/non-decreasing order.
22. There are many problems which reduce to the this problem such as box stacking and the building bridges. These days the interviewers expect an $N\log N$ solution.
23. **Edit Distance** Given two strings and a set of operations Change (C), insert (I) and delete (D) , find minimum number of edits (operations) required to transform one string into another.
24. **0/1 Knapsack Problem** A thief robbing a store and can carry a maximal weight of W into their knapsack. There are n items and i th item weigh w_i and is worth v_i dollars. What items should thief take?
25. **Balanced Partition** You have a set of n integers each in the range $0 \leq K$. Partition these integers into two subsets such that you minimize $|S_1 - S_2|$

S_2 —, where S_1 and S_2 denote the sums of the elements in each of the two subsets.

26. **Coin Change** Given a value N , if we want to make change for N cents, and we have infinite supply of each of $S = S_1, S_2, \dots, S_m$ valued coins, how many ways can we make the change?
27. **Longest Common Subsequence** Find the longest common subsequence of two strings A and B where the elements are letters from the two strings and they should be in the same order.
28. **Longest Palindromic Subsequence** The question is same as above but the subsequence should be palindromic as well.
29. **Minimum Number of Jumps** Given an array of integers where each element represents the maximum number of steps that can be made forward from that element, find the minimum number of jumps to reach the end of the array (starting from the first element).

1.5 Brian Dean, MIT

30. **Maximum Value Contiguous Subsequence** Given a sequence of n real numbers $A(1) \dots A(n)$, determine a contiguous subsequence $A(i) \dots A(j)$ for which the sum of elements in the subsequence is maximized.
31. **Making Change** You are given n types of coin denominations of values $v(1) ; v(2) ; \dots ; v(n)$ (all integers). Assume $v(1) = 1$, so you can always make change for any amount of money C . Give an algorithm which makes change for an amount of money C with as few coins as possible. [on problem set 4]
32. **Longest Increasing Subsequence** Given a sequence of n real numbers $A(1) \dots A(n)$, determine a subsequence (not necessarily contiguous) of maximum length in which the values in the subsequence form a strictly increasing sequence. [on problem set 4]
33. **Box Stacking**. You are given a set of n types of rectangular 3-D boxes, where the i -th box has height $h(i)$, width $w(i)$ and depth $d(i)$ (all real numbers). You want to create a stack of boxes which is as tall as possible, but you can only stack a box on top of another box if the dimensions of the 2-D base of the lower box are each strictly larger than

those of the 2-D base of the higher box. Of course, you can rotate a box so that any side functions as its base. It is also allowable to use multiple instances of the same type of box.

34. **Building Bridges** Consider a 2-D map with a horizontal river passing through its center. There are n cities on the southern bank with x -coordinates $a(1) \dots a(n)$ and n cities on the northern bank with x -coordinates $b(1) \dots b(n)$. You want to connect as many north-south pairs of cities as possible with bridges such that no two bridges cross. When connecting cities, you can only connect city i on the northern bank to city i on the southern bank. (Note: this problem was incorrectly stated on the paper copies of the handout given in recitation.)
35. **Integer Knapsack Problem (Duplicate Items Forbidden)** This is the same problem as the example above, except here it is forbidden to use more than one instance of each type of item.
36. **Balanced Partition.** You have a set of n integers each in the range $0 \dots K$. Partition these integers into two subsets such that you minimize $|S_1 - S_2|$, where S_1 and S_2 denote the sums of the elements in each of the two subsets.
37. **Edit Distance** Given two text strings A of length n and B of length m , you want to transform A into B with a minimum number of operations of the following types: delete a character from A , insert a character into A , or change some character in A into a new character. The minimal number of such operations required to transform A into B is called the edit distance between A and B .
38. **Counting Boolean Parenthesizations** You are given a boolean expression consisting of a string of the symbols 'true', 'false', 'and', 'or', and 'xor'. Count the number of ways to parenthesize the expression such that it will evaluate to true. For example, there are 2 ways to parenthesize 'true and false xor true' such that it evaluates to true.
39. **Optimal Strategy for a Game** Consider a row of n coins of values $v(1) \dots v(n)$, where n is even. We play a game against an opponent by alternating turns. In each turn, a player selects either the first or last coin from the row, removes it from the row permanently, and receives the value of the coin. Determine the maximum possible amount of money we can definitely win if we move first.

Antonio Gulli

1. Nothing yet.

1.6 Brian Dean, Clemson University

Dynamic Programming Practice Problems

This site contains an old collection of practice dynamic programming problems and their animated solutions that I put together many years ago while serving as a TA for the undergraduate algorithms course at MIT. I am keeping it around since it seems to have attracted a reasonable following on the web. Eventually, this animated material will be updated and incorporated into an algorithms textbook I am writing. – Brian Dean

To view the solution to one of the problems below, click on its title. To view the solutions, you'll need a machine which can view Macromedia Flash animations and which has audio output. I have also included a short review animation on how to solve the integer knapsack problem (with multiple copies of items allowed) using dynamic programming.

Problems:

40. **Maximum Value Contiguous Subsequence.** Given a sequence of n real numbers $A(1) \dots A(n)$, determine a contiguous subsequence $A(i) \dots A(j)$ for which the sum of elements in the subsequence is maximized.
41. **Making Change.** You are given n types of coin denominations of values $v(1) ; v(2) ; \dots ; v(n)$ (all integers). Assume $v(1) = 1$, so you can always make change for any amount of money C . Give an algorithm which makes change for an amount of money C with as few coins as possible. [on problem set 4]
42. **Longest Increasing Subsequence.** Given a sequence of n real numbers $A(1) \dots A(n)$, determine a subsequence (not necessarily contiguous) of maximum length in which the values in the subsequence form a strictly increasing sequence. [on problem set 4]
43. **Box Stacking.** You are given a set of n types of rectangular 3-D boxes, where the i -th box has height $h(i)$, width $w(i)$ and depth $d(i)$ (all real numbers). You want to create a stack of boxes which is as tall as possible, but you can only stack a box on top of another box if the dimensions of the 2-D base of the lower box are each strictly larger than

those of the 2-D base of the higher box. Of course, you can rotate a box so that any side functions as its base. It is also allowable to use multiple instances of the same type of box.

44. **Building Bridges.** Consider a 2-D map with a horizontal river passing through its center. There are n cities on the southern bank with x -coordinates $a(1) \dots a(n)$ and n cities on the northern bank with x -coordinates $b(1) \dots b(n)$. You want to connect as many north-south pairs of cities as possible with bridges such that no two bridges cross. When connecting cities, you can only connect city i on the northern bank to city i on the southern bank. (Note: this problem was incorrectly stated on the paper copies of the handout given in recitation.)
45. **Integer Knapsack Problem (Duplicate Items Forbidden).** This is the same problem as the example above, except here it is forbidden to use more than one instance of each type of item.
46. **Balanced Partition.** You have a set of n integers each in the range $0 \dots K$. Partition these integers into two subsets such that you minimize $|S_1 - S_2|$, where S_1 and S_2 denote the sums of the elements in each of the two subsets.
47. **Edit Distance.** Given two text strings A of length n and B of length m , you want to transform A into B with a minimum number of operations of the following types: delete a character from A , insert a character into A , or change some character in A into a new character. The minimal number of such operations required to transform A into B is called the edit distance between A and B .
48. **Counting Boolean Parenthesizations.** You are given a boolean expression consisting of a string of the symbols 'true', 'false', 'and', 'or', and 'xor'. Count the number of ways to parenthesize the expression such that it will evaluate to true. For example, there are 2 ways to parenthesize 'true and false xor true' such that it evaluates to true.
49. **Optimal Strategy for a Game.** Consider a row of n coins of values $v(1) \dots v(n)$, where n is even. We play a game against an opponent by alternating turns. In each turn, a player selects either the first or last coin from the row, removes it from the row permanently, and receives the value of the coin. Determine the maximum possible amount of money we can definitely win if we move first.